



汇编语言

12 子程序与参数传递

张青

ieqzhang@zzu.edu.cn

实验六 循环程序设计

- 1、编写汇编语言程序 exp0601.s。要求：使用循环结构，进行自然数相加（ $1 + 2 + 3 + \dots + N$ ）。无符号整数的累加和用一个32位寄存器存储，要求程序求出（显示）有效累加和的最大值及对应的最大N值。
运行结果用两行显示，累加和在第一行显示，N值在第二行显示。
- 2、编写汇编语言程序 exp0602.s。将斐波那契数列程序编写为完整的汇编语言源程序。要求：输出斐波那契数，一行一个，直到超出32位数据范围不能表达为止。
- 3、编写汇编语言程序 exp0603.s。将多重循环的冒泡法排序程序编写为完整的汇编语言源程序。
- 4、编写汇编语言程序 exp0604.s。显示ASCII表程序，要求仅在数据段设置表格缓冲区，编程将ASCII代码值填入留出位置的表格，然后调用显示功能实现（需要利用双重循环）。

```
0000000000
00000
```

```
1
1
2
3
5
.....
```

```
10 | 0 1 2 3 4 5 6 7 8 9 A B C D E F
-- + -----
20 | ! " # $ % & ' ( ) * + , - . /
30 | 0 1 2 3 4 5 6 7 8 9 : ; < = > ?
40 | @ A B C D E F G H I J K L M N O
50 | P Q R S T U V W X Y Z [ \ ] ^ _
60 | ` a b c d e f g h i j k l m n o
70 | p q r s t u v w x y z { | } ~
```

- 子程序结构
- 子程序调用与返回指令

教材章节：5.3.1-5.3.2

慕课：9.2

- ▶ **call指令**
- ▶ **ret指令**
- ▶ **子程序的结构**

- 子程序设计
- 参数传递方式

教材章节：5.3.3-5.3.5

慕课：9.2-9.3

- ▶ **堆栈的作用**
- ▶ **寄存器传送**
- ▶ **共享变量传递**
- ▶ **堆栈传递**

学习目标



- 熟记call 和ret指令的功能
- 能画出堆栈在子程序调用和返回中的作用
- 会设计子程序，并调用
- 能说出子程序调用过程中RIP寄存器的变化
- 会使用三种参数传递方式
- 会阅读分析子程序结构程序的功能、运行结果
- 会用gdb调试子程序执行过程
- 能说出程序开发过程中出现问题的原因并解决
- 养成良好的编程习惯，养成细心、严谨的工作态度
- 养成团队合作，互相帮助的精神

汇编子程序结构



.text

子程序名: #子程序开始
#子程序体
ret #子程序返回

main: #程序入口
#主程序结束

子程序名: #子程序开始
#子程序体
ret #子程序返回

〔例5-18〕子程序结构-C语言源程序



```
#include <stdio.h>
int mult2(int x, int y){
    int t = x*y;
    return t;
}
void multstore(int x, int y, int *dest) {
    int t = mult2(x, y);
    *dest = t;
}
int main(){
    int m;
    multstore(3,5,&m);
    printf("3 * 5 =%d",m);
    return 0;
}
```

1. 子程序调用指令CALL

- CALL指令用在主程序中，实现子程序的调用
 - 分成段内调用（近调用）和段间调用（远调用）
 - 64位模式下，目标地址采用相对寻址或间接寻址
 - 32位兼容模式下，目标地址可以采用相对寻址、间接寻址、或直接寻址
- CALL指令的两个功能：
 - 入栈返回地址：将CALL下条指令的地址压入堆栈
 - 目标地址送RIP $RSP \leftarrow RSP - 8, SS:[RSP] \leftarrow RIP, RIP \leftarrow \text{目标地址}$
 - 实模式：16位，32位保护模式：32位

GDB反汇编-main函数



Dump of assembler code for function main:

0x00007ff72918172b <+0>: 48 83 ec 38	sub rsp,0x38
0x00007ff72918172f <+4>: e8 f3 00 00 00	call 0x7ff729181827 <__main>
0x00007ff729181734 <+9>: 4c 8d 44 24 2c	lea r8,[rsp+0x2c]
0x00007ff729181739 <+14>: ba 05 00 00 00	mov edx,0x5
0x00007ff72918173e <+19>: b9 03 00 00 00	mov ecx,0x3
0x00007ff729181743 <+24>: e8 ce ff ff ff	call 0x7ff729181716 <multstore>
0x00007ff729181748 <+29>: 8b 54 24 2c	mov edx,DWORD PTR [rsp+0x2c]
0x00007ff72918174c <+33>: 48 8d 0d fd 38 00 00	lea rcx,[rip+0x38fd] # 0x7ff729185050
0x00007ff729181753 <+40>: e8 38 17 00 00	call 0x7ff729182e90 <printf>
0x00007ff729181758 <+45>: b8 00 00 00 00	mov eax,0x0
0x00007ff72918175d <+50>: 48 83 c4 38	add rsp,0x38
0x00007ff729181761 <+54>: c3	ret

set disassembly-flavor intel
disas /r

目标地址 = 0x00007ff729181748 + 0xFFFFFCE
= 0x00007ff729181716

GDB反汇编-multstore和mult2函数



Dump of assembler code for function multstore:

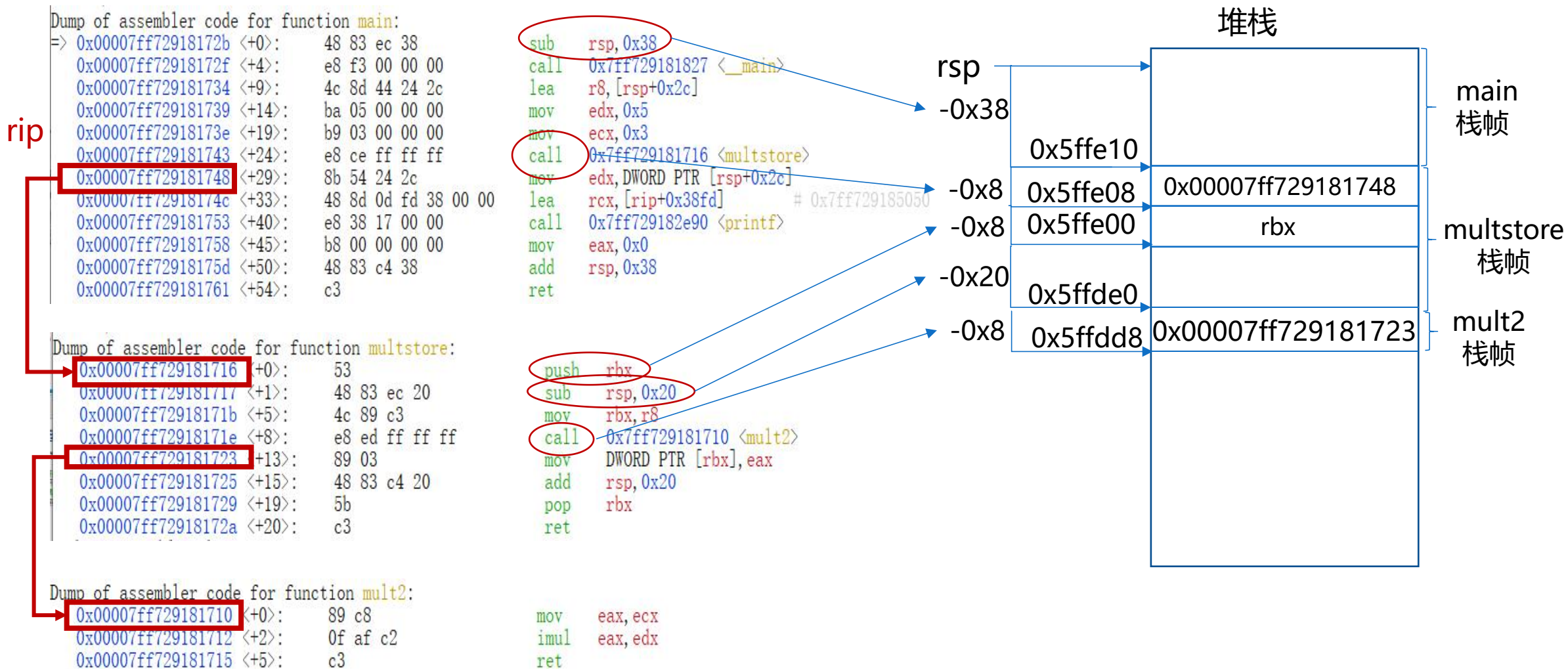
```
0x00007ff729181716 <+0>: 53          push  rbx
0x00007ff729181717 <+1>: 48 83 ec 20  sub  rsp,0x20
0x00007ff72918171b <+5>: 4c 89 c3     mov   rbx,r8
0x00007ff72918171e <+8>: e8 ed ff ff  call  0x7ff729181710 <mult2>
0x00007ff729181723 <+13>: 89 03       mov   DWORD PTR [rbx],eax
0x00007ff729181725 <+15>: 48 83 c4 20  add   rsp,0x20
0x00007ff729181729 <+19>: 5b         pop   rbx
0x00007ff72918172a <+20>: c3         ret
```

Dump of assembler code for function mult2:

```
0x00007ff729181710 <+0>: 89 c8       mov   eax,ecx
0x00007ff729181712 <+2>: 0f af c2    imul  eax,edx
0x00007ff729181715 <+5>: c3         ret
```

目标地址= 0x00007ff729181723 + 0xFFFFFED
= 0x00007ff729181710

eg0518调用过程



2. 子程序返回指令RET

- RET指令用在子程序结束，实现返回主程序

RET

#无参数返回：出栈返回地址

RET i16

#有参数返回：① 出栈返回地址

② $RSP \leftarrow RSP + i16$

汇编程序会根据存储模型等信息确定子程序的远近调用，并相应产生返回指令

eg0518返回过程



Dump of assembler code for function `main`:

```
=> 0x00007ff72918172b <+0>: 48 83 ec 38
0x00007ff72918172f <+4>: e8 f3 00 00 00
0x00007ff729181734 <+9>: 4c 8d 44 24 2c
0x00007ff729181739 <+14>: ba 05 00 00 00
0x00007ff72918173e <+19>: b9 03 00 00 00
0x00007ff729181743 <+24>: e8 ce ff ff ff
0x00007ff729181748 <+29>: 8b 54 24 2c
0x00007ff72918174c <+33>: 48 8d 0d fd 38 00 00
0x00007ff729181753 <+40>: e8 38 17 00 00
0x00007ff729181758 <+45>: b8 00 00 00 00
0x00007ff72918175d <+50>: 48 83 c4 38
0x00007ff729181761 <+54>: c3
```

```
sub    rsp,0x38
call   0x7ff729181827 <__main>
lea    r8,[rsp+0x2c]
mov    edx,0x5
mov    ecx,0x3
call   0x7ff729181716 <multstore>
mov    edx,DWORD PTR [rsp+0x2c]
lea    rcx,[rip+0x38fd] # 0x7ff729185058
call   0x7ff729182e90 <printf>
mov    eax,0x0
add    rsp,0x38
ret
```

Dump of assembler code for function `multstore`:

```
0x00007ff729181716 <+0>: 53
0x00007ff729181717 <+1>: 48 83 ec 20
0x00007ff72918171b <+5>: 4c 89 c3
0x00007ff72918171e <+8>: e8 ed ff ff ff
0x00007ff729181723 <+13>: 89 03
0x00007ff729181725 <+15>: 48 83 c4 20
0x00007ff729181729 <+19>: 5b
0x00007ff72918172a <+20>: c3
```

```
push   rbx
sub    rsp,0x20
mov    rbx,r8
call   0x7ff729181710 <mult2>
mov    DWORD PTR [rbx],eax
add    rsp,0x20
pop    rbx
ret
```

Dump of assembler code for function `mult2`:

```
0x00007ff729181710 <+0>: 89 c8
0x00007ff729181712 <+2>: 0f af c2
0x00007ff729181715 <+5>: c3
```

```
mov    eax,ecx
imul   eax,edx
ret
```

堆栈

+0x38

0x5ffe10

+0x8

0x5ffe08

0x00007ff729181748

+0x8

0x5ffe00

rbx

+0x20

0x5ffde0

+0x8

0x5ffdd8

0x00007ff729181723

rsp

RET指令不能省略



mult2:

0x00007ff729181710 <+0>: 89 c8 mov eax,ecx

0x00007ff729181712 <+2>: 0f af c2 imul eax,edx

0x00007ff729181715 <+5>: c3 ret

multstore:

0x00007ff729181716 <+0>: 53 push rbx

0x00007ff729181717 <+1>: 48 83 ec 20 sub rsp,0x20

.....

0x00007ff72918171e <+8>: e8 ed ff ff ff call 0x7ff729181710 <mult2>

0x00007ff729181723 <+13>: 89 03 mov DWORD PTR [rbx],eax

如果子程序缺少ret指令可能会导致意想不到的结果
因此，必须使用ret指令从子程序显式返回。

子程序设计

- 子程序的编写方法与主程序一样
- 但需要留意几个问题：
 - ✓ 子程序开始要用标号表示子程序名
 - ✓ RET指令返回主程序，CALL指令调用子程序
 - ✓ 压入和弹出操作要成对使用，保持堆栈平衡
 - ✓ 开始保护寄存器，返回前相应恢复
 - ✓ 安排在代码段的主程序之外，程序入口之前，或程序返回之后
 - ✓ 子程序允许嵌套和递归
 - ✓ 为与其他语言编写的模块共同使用则应遵循ABI规范

最好有完整的注释

难点是参数传递和栈帧管理

〔例5-19〕回车换行子程序



dpcrlf:	#回车换行子程序
push rax	#保护寄存器
mov al,0dh	#输出回车字符
call dispc	#子程序中调用子程序，实现子程序嵌套
mov al,0ah	#输出换行字符
call dispc	
pop rax	#恢复寄存器
ret	#子程序返回

〔例5-20〕子程序递归-计算n!



fact:

<code>push rbx</code>	#保存rbx
<code>mov ebx, ecx</code>	#入口参数rcx: n
<code>test ecx, ecx</code>	
<code>je .L1</code>	# n=0时, 返回1
<code>.L4: lea ecx, -1[rcx]</code>	# n不等于0, 则计算n*fact(n-1)
<code>call fact</code>	#递归调用fact
<code>imul eax, ebx</code>	#fact (n-1) 返回, n*fact (n-1)
<code>jmp .L2</code>	
<code>.L1: mov eax, 1</code>	# 返回值在eax
<code>.L2: pop rbx</code>	#恢复rbx
<code>ret</code>	#函数返回

下面指令序列的输出结果是什么？

```
mov eax 123  
call dispsid  
call dispctrlf  
call dispsid
```

参数传递

- 主程序与子程序间通过参数传递建立联系
 - 入口参数(输入参数): 主程序→子程序
 - 出口参数(输出参数): 子程序→主程序
- 传递参数的多少反映程序模块间的耦合程度
- 参数的具体内容
 - 数据本身 (传递数值)
 - 数据的存储地址 (传递地址, 传递引用)

参数传递方法

- ✓通用寄存器
- ✓共享变量
- ✓堆栈

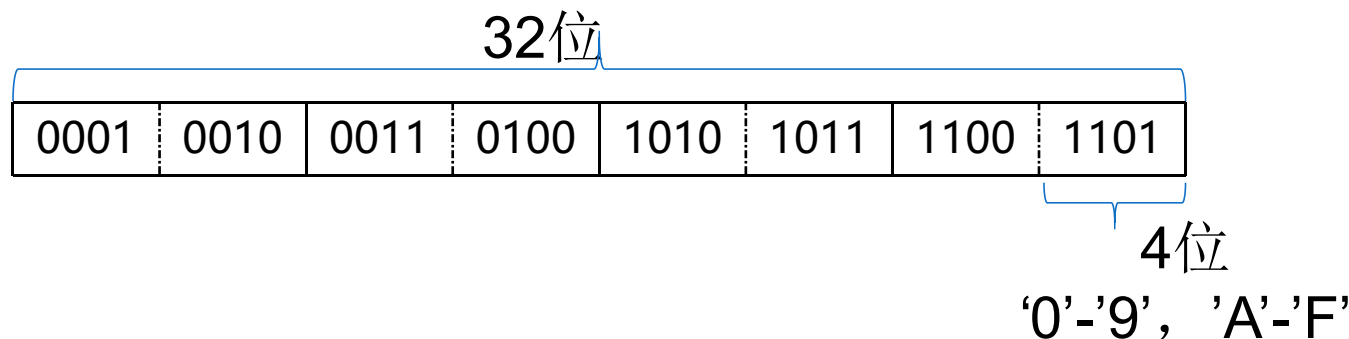
寄存器传递参数

- 最简单和常用的参数传递方法
- 把参数存于约定的寄存器
 - 少量数据直接传递数值
 - 大量数据只能传递地址
- 带有出口参数的寄存器不能保护和恢复
- 带有入口参数的寄存器可以保护、也可以不保护，但最好能够保持一致

〔例5-21〕 十六进制显示程序



利用字符串显示子程序DISPMMSG实现32位数的十六进制显示。



转换的算法

- AL的低四位加0x30转换为ASCII码
- 如果大于字符 '9' 的ASCII码, 再加7, 转换为 'A' - 'F' 的ASCII码

```
.data                #定义数据段
regd: .ascii "EAX="  #显示EAX内容,
      .space 8        #预留8个字符 (字节) 空间
      .byte  '\n' ,0  #回车换行, 以0结尾
```

〔例5-21〕 十六进制显示程序-1



.text	#定义代码段
.globl main	
main:	# 主程序
mov eax, 0x1234abcd	#假设一个要显示的数据
xor rbx,rbx	#使用rbx相对寻址访问REGD字符串
lea rdi,regd[rip]	#rdi保存显示字符串首地址
mov ecx,8	# 8位十六进制数
again: rol eax,4	#高4位循环移位进入低4位
push rax	#保存EAX中的数据
call htoasc	#调用子程序
mov 4[rdi+rbx],al	#保存转换后的ASCII码
pop rax	#恢复保存的数据
inc rbx	#下一个字符
loop again	

〔例5-21〕 十六进制显示程序-2



lea rax,regd[rip] #等价于 mov rax, offset regd[rip]

call dispmsg #显示

mov eax,0

ret # 主程序执行结束

#子程序

htoasc: #将AL低4位表达的一位十六进制数转换为ASCII码

and al,0xf #只取AL的低4位

or al,0x30 #AL高4位变成3，实现加30H

cmp al,0x39 #是0 ~ 9，还是A ~ F

jbe htoend

add al,7 #是A ~ F，其ASCII码再加上7

htoend: ret #子程序返回

〔例5-22〕 有符号十进制显示程序



以十进制形式显示32位二进制数的真值

- **转换的算法如下：**

- 1) 首先判断数据是零、正数或负数，是零显示 “0” 退出
- 2) 是负数，显示负号 “-”，求数据的绝对值
- 3) 接着数据除以10，余数为十进制数码，加0x30转换为ASCII码保存
- 4) 重复第（3）步，直到商为0结束
- 5) 依次从高位开始显示各位数字

$$\begin{array}{rcl} 123 \div 10 & = & 12 \dots\dots 3 (0x33) \\ 12 \div 10 & = & 1 \dots\dots 2 (0x32) \\ 1 \div 10 & = & 0 \dots\dots 1 (0x31) \end{array} \quad \uparrow$$



共享变量传递参数

- 子程序和主程序使用同一个变量名存取数据
- 如果变量定义和使用不在同一个程序模块中
 - 需要利用`.global`、`.extern`声明
- 共享变量传递参数，子程序的通用性较差
- 特别适合在多个程序段间、尤其在不同的程序模块间传递数据

〔例5-23〕 二进制数字字符串转换为二进制数程序



- 将二进制数字字符串转换为二进制数的功能编写成一个子程序，用共享变量返回结果。

转换算法：

- 字符 “0” 或 “1” 减去0x30转换成数值 “0” 或 “1” 。
- 将原数值左移1位，并与新数值进行组合。
- 重复转换每个字符
- 子程序入口参数rdx：二进制数字字符串的首地址
- 出口参数是共享变量temp

〔例5-23〕 二进制数字字符串转换为数主程序



```
        .data        #定义数据段
temp: .space 4        #共享变量, 32位数
Binstr: .asciz "01111111"
        .text        #定义代码段
        .globl main
main:                # 程序执行起始位置
        lea rdx, Binstr[rip]        #入口参数rdx
        call BtoD
        mov eax,temp[rip]        #获得出口参数
        call disphd
        mov eax, 0
        ret
```

〔例5-23〕 二进制数字字符串转换为数值子程序-1



```
BtoD:  #二进制字符串转换为双字数值子程序
        push rax  #入口参数: rdx, 出口参数: 共享变量TEMP
        push rbx
        push rcx
        push rdx
rdbd1:  ....      #转换

        pop rdx
        pop rcx
        pop rbx
        pop rax
        ret
```

〔例5-23〕 二进制数字字符串转换为数字程序-2



```
rdbd1:  xor ebx,ebx      #rbx用于存放二进制结果
        mov ecx,32     #限制最多字符的个数
rdbd2:  mov al,[rdx]     #输入一个字符
        cmp al,0
        jz L1          #字符串以0结尾
        sub al,'0'     #对输入的字符进行转化
        shl ebx,1      #EBX的值左移一位
        or bl,al       #BL和AL相加
        inc rdx
        loop rdbd2     #取下一个字符
L1:     mov temp[rip],ebx #把EBX的二进制结果存放TEMP返回
```

【例5-24】有符号十进制数字串转换为有符号数程序



十进制数字串转换为补码的算法

- (1) 首先判断字符串长度是否在32位数表示范围内，然后判断输入了正数（正号引导）还是负数（负号引导），无符号引导也是正数，可以用一个寄存器记录下来。
- (2) 接着判断下一个字符是否为有效数码：0~9（ASCII码）。字符无效，提示错误信息，并设置标志位 $CF=1$ ，子程序返回。字符有效，继续。
- (3) 字符有效，减 $0x30$ 转换为二进制数；然后将前面输入的数值乘10，并与刚输入的数字相加得到新的数值。因为刚输入的数码作为十进制个位，则前面输入的数值依次向十位、百位等移动一位，即乘以10。
- (4) 判断输入的数据是否超出了有效范围。超出范围，提示错误信息，并设置标志位 $CF=1$ ，子程序返回。没有超出范围，则继续。
- (5) 重复（2）~（4）步，如果输入的字符都有效，则一直处理完。
- (6) 如果是负数进行求补，转换成补码；否则直接将数值保存，设置标志位 $CF=0$ ，子程序返回。

有符号十进制数字串转换为有符号数

